

Week 7 Lecture

Welcome to the Week 7 Lecture. This is a turning point in the course, we hope you enjoy it. Today we will:

- Take a look at programming libraries and frameworks.
- Introduce p5.js.
- Learn how p5.js is structured.
- Introduce the major assignment for this unit.

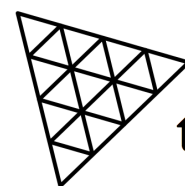
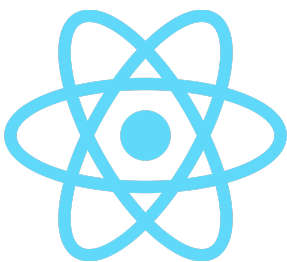
1. What is a Programming Library

Introduction

What is a programming library?

- **A toolkit:** Think of it as a toolbox filled with tools (functions) that help you accomplish tasks without building tools from scratch. This is something like the `utils.js` file we have been making, except a lot bigger with a lot more useful abstractions.
- **Difference between a library and a framework.**
 - **Library:** It is a collection of related functions, methods, and objects towards a specific functionality. The programmer can choose what to use when.
 - **Framework:** It is a collection of rules, structure, functions, methods, objects, but contains everything needed to build an application, and the programmer has to follow strict rules and structure within the framework to build the application.
- We will primarily focus on libraries.

2. Popular JavaScript Frameworks and Libraries



three.js

Libraries:

- [React.js](https://react.dev/)  [\(https://react.dev/\)](https://react.dev/): Building user interfaces.

- Component-driven: Think of building a UI like using LEGO blocks
- **Three.js**  [\(https://threejs.org/\)](https://threejs.org/): 3D graphics.
 - Bring 3D content to the web without any plugins.
- **D3.js**  [\(https://d3js.org/\)](https://d3js.org/): Data-driven documents for visualisations.
 - Bind data to graphics and apply transformations.

There are also many many many other more focused javascript libraries that facilitate specific behaviour some examples are:

- **Leaflet.js**  [\(https://leafletjs.com/\)](https://leafletjs.com/): A javascript library for showing and manipulating maps.
- **noUISlider**  [\(https://refreshless.com/nouislider/\)](https://refreshless.com/nouislider/): A javascript library for creating sliders.
- **Moment.js**  [\(https://momentjs.com/\)](https://momentjs.com/): A library for parsing, validating, manipulating, and displaying dates and times in JavaScript.

Frameworks:

- **Angular.js**  [\(https://angularjs.org/\)](https://angularjs.org/): Web application framework.
 - MVC architecture: Model, View, Controller.
- **Vue.js**  [\(https://vuejs.org/\)](https://vuejs.org/): Progressive framework for building user interfaces.
 - Virtual DOM: A lightweight copy of the actual DOM, faster updates.

3. Introducing P5.js

What is P5.js?

- P5.js is a javascript library that wraps a lot of useful functions, like sound, video, animation and more.
- A spiritual successor to Processing, a language designed for artists and visual designers. P5.js brings that philosophy to the web.

P5.js has a lot of active users and is used a lot in education, you can find a huge amount of information and resources online.

Some resources we think are great are:

- The p5.js reference site <https://p5js.org/reference/>  [\(https://p5js.org/reference/\)](https://p5js.org/reference/) this is the documentation that tells you about all the built in functionality

- Other p5.js website resources like the examples <https://p5js.org/examples/> and the tutorials page <https://p5js.org/tutorials/> . <https://p5js.org/learn/>
- [The coding train](https://www.youtube.com/channel/UCvjgXvBlbQiydffZU7m1_aw) . https://www.youtube.com/channel/UCvjgXvBlbQiydffZU7m1_aw

Structure of a P5.js sketch

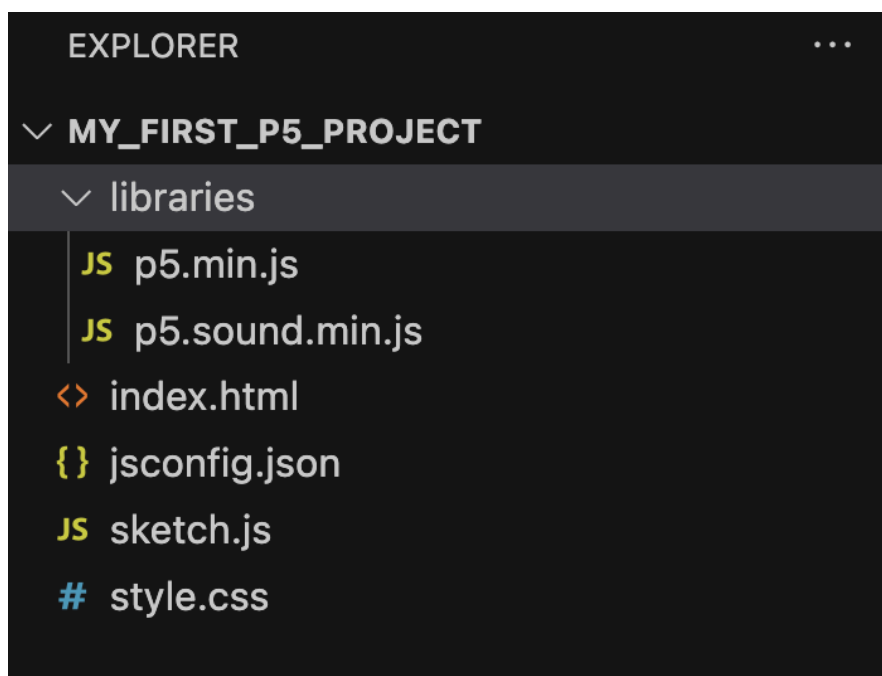
We call a 'project' or a 'page' in p5.js a 'sketch'.

We will be using visual studio code and the p5.vscode plugin to work with the p5.js library. When we use the p5.vscode library to make a new project it will automatically create and organise everything we need.

Like the pages that we have been working on there is

- An html file called index.html.
- A css file called style.css.
- And some javascript files, one called sketch.js and then the p5.js library files (p5.min.js and p5.sound.min.js, these are stored in a folder called libraries).

After creating a new project with the plugin our VSCode project folder will look like this:



The html file is the 'entry point' as in our previous work and it has the following structure:

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="utf-8" />
```

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Sketch</title>
<link rel="stylesheet" type="text/css" href="style.css">
<script src="libraries/p5.min.js"></script>
<script src="libraries/p5.sound.min.js"></script>
</head>
<body>
  <script src="sketch.js"></script>
</body>
</html>
```

You can see that there are three script tags.

- One for the p5.js library
- One for the p5.sound library and
- One for a sketch.js file.

We will do almost all our work in the sketch.js file.

The sketch.js file

The p5 libraries take care of most of the work for us, and interpret the code we write in the sketch.js file in a reliable way.

You will see that the sketch.js file made from the template has two functions in it, setup() and draw().

- setup() is run once when the sketch is loaded. It is used to set up the canvas, and any other initialisation.
- draw() is run continuously, and is used to draw the contents of the sketch. We can set how many times per second the draw function is run, or we can make it only run once. Without specifying anything the draw function will run 60 times per second.

This is what the templated p5.js sketch.js file looks like:

```
function setup() {
  createCanvas(400, 400);
}

function draw() {
  background(220);
}
```

The draw() function is, as it sounds, used to draw the contents of the sketch. Because it updates continuously we can easily animate our sketches. p5.js uses a canvas to draw on instead of the svg container we have been using. The canvas is a standard html element, to set up a canvas using vanilla javascript we would do something like this:

```
// Create a canvas element
let canvas = document.createElement('canvas');
```

```
// Set the canvas dimensions
canvas.width = 600; // Width of the canvas
canvas.height = 400; // Height of the canvas

// Append the canvas to the body of the document
document.body.appendChild(canvas);

// Get the 2D drawing context
let ctx = canvas.getContext('2d');

// Now you can use the 'ctx' variable to draw on the canvas
```

But in p5.js we can just call the `createCanvas()` function (like in the template). This function takes two arguments, the width and height of the canvas. The templated `sketch.js` file has the canvas size set to 400 by 400 pixels.

The templated sketch also has `background(220)` in the `draw()` function. This sets the background colour of the canvas to a light grey.

In p5.js we can describe colours with a single number if they are greyscale (0 is black and 255 is white), or we can use three numbers to describe a colour in RGB format. we can also use strings (words) or hex values to define colours ([here](https://p5js.org/reference/#/p5/color)  <https://p5js.org/reference/#/p5/color> is the colour reference information).

Drawing basic shapes

We can draw basic shapes like rectangles, circles, lines and triangles using the p5.js library. They have templates for all of these shapes in the [p5.js reference](https://p5js.org/reference/)  <https://p5js.org/reference/> some basic prototypes are:

Draw a rectangle:

```
rect(x, y, width, height);
```

Draw a circle:

```
circle(x, y, radius);
```

Draw an ellipse

```
ellipse(x, y, width, height);
```

Draw a line

```
line(x1, y1, x2, y2);
```

Draw a triangle

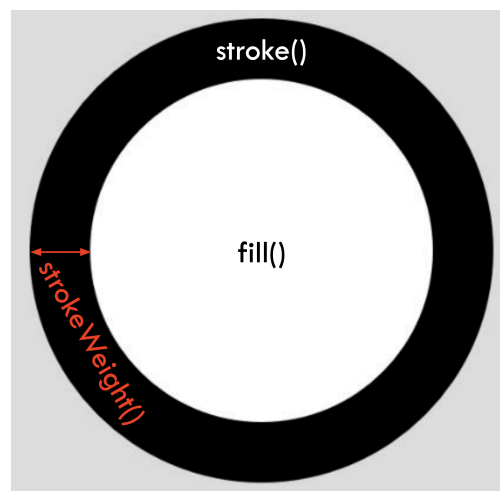
```
triangle(x1, y1, x2, y2, x3, y3);
```

Styling functions

Styling functions allow you to set the colour of the fill and stroke of drawing commands as well as setting the thickness of lines and turning off the fill and/or stroke entirely. We have so far mostly been using these by calling `setAttribute` on an object in javascript. In p5.js these calls are simpler.

These functions are:

- `fill()` - for setting the colour of the fill.
- `stroke()` - for setting the colour of the stroke.
- `strokeWeight()` - for setting the thickness of the stroke.
- `noFill()` - for turning fill off.
- `noStroke()` - for turning stroke off.



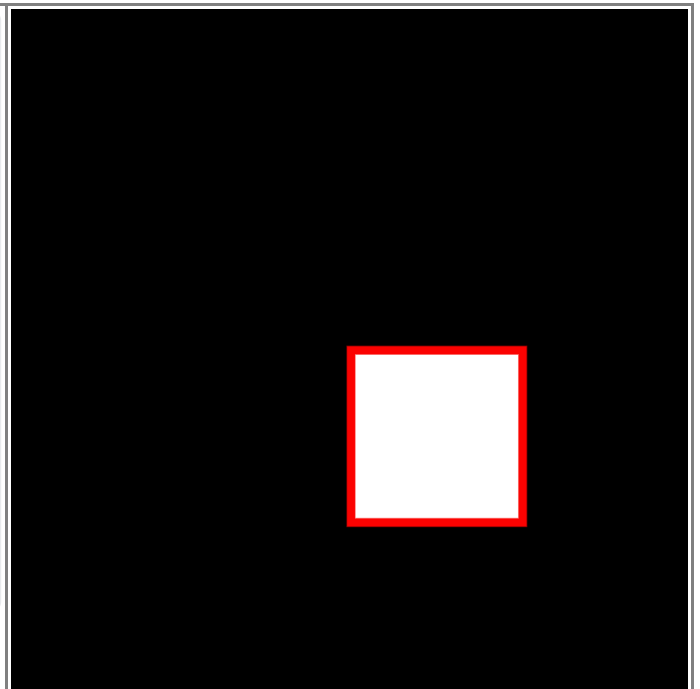
Order matters!

In p5.js, the order in which you call styling functions and drawing functions is crucial. Styling functions set the style properties for all subsequent drawing commands. Once a style is set, every shape or element drawn after that will inherit that style until a new style is set.

The following code and comments show these in use:

```
function setup() {
  //lets create a canvas of 400x400 pixels
  createCanvas(400, 400);
}

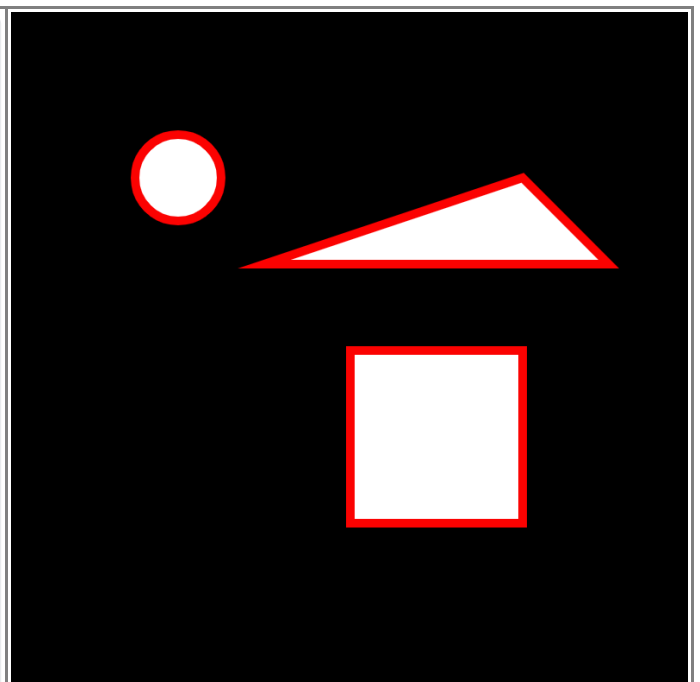
function draw() {
  //lets set the background to black
  background(0);
  //lets set the fill to white
  fill(255);
  //lets set the stroke to red
  stroke(255, 0, 0);
  //lets set the stroke weight to 5
  strokeWeight(5);
  //lets draw a rectangle at 200,200 with a
  width and height of 100
  rect(200, 200, 100, 100); //rect(x,y,widt
h,height)
}
```



If we draw other shapes after the rectangle they will all have the same style applied.

```
function setup() {
  //lets create a canvas of 400x400 pixels
  createCanvas(400, 400);
}

function draw() {
  //lets set the background to black
  background(0);
  //lets set the fill to white
  fill(255);
  //lets set the stroke to red
  stroke(255, 0, 0);
  //lets set the stroke weight to 5
  strokeWeight(5);
  //lets draw a rectangle at 200,200 with a
  width and height of 100
  rect(200, 200, 100, 100); //rect(x,y,widt
h,height)
  circle(100, 100, 50); //rect(x,y,width,he
ight)
  triangle(300, 100, 150, 159, 350, 150);
  //trianglle(x1,y1,x2,y2,x3,y3)
}
```

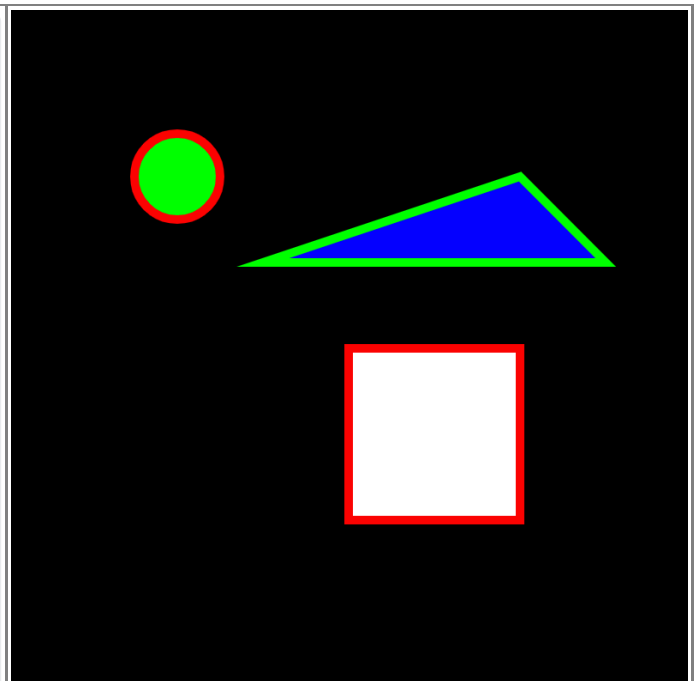


Overriding Styles

You can override styles by setting a new style after a drawing command. For example, if you set a fill color, draw a shape, and then set a different fill colour, the next shape you draw will have the new fill colour, while the previous shape retains the original fill colour.

Take a look at this example, and note that styles that were not changed will be persistent, even though some of the styles are changed, for example the stroke weight stays the same the whole time as it is not specifically set to another value.

```
function setup() {  
  //lets create a canvas of 400x400 pixels  
  createCanvas(400, 400);  
}  
  
function draw() {  
  //lets set the background to black  
  background(0);  
  //lets set the fill to white  
  fill(255);  
  //lets set the stroke to red  
  stroke(255, 0, 0);  
  //lets set the stroke weight to 5  
  strokeWeight(5);  
  //lets draw a rectangle at 200,200 with a width and height of 100  
  rect(200, 200, 100, 100); //rect(x,y,width,height)  
  
  //lets change the fill colour to green  
  fill(0, 255, 0);  
  circle(100, 100, 50); //circle(x,y,radius)  
  
  //lets change the fill colour to blue;  
  fill(0, 0, 255);  
  //lets change the stroke colour to green  
  stroke(0, 255, 0);  
  triangle(300, 100, 150, 150, 350, 150); //triangle(x1, y1, x2, y2, x3, y3)  
}
```



There are more advanced methods for managing style we will look at later in the course, for now this will help you to control the colours and styles of the shapes we draw.

Built in variables

P5.js has some built in variables that are useful, some of these are:

width and height

These correspond to the canvas size, if we were to set our canvas to 600 pixels by 350 pixels in our setup like this

```
createCanvas(600, 350);
```

- width will be equal to 600
- height will be equal to 350

This is useful because it allows us to know the boundaries of our drawing area.

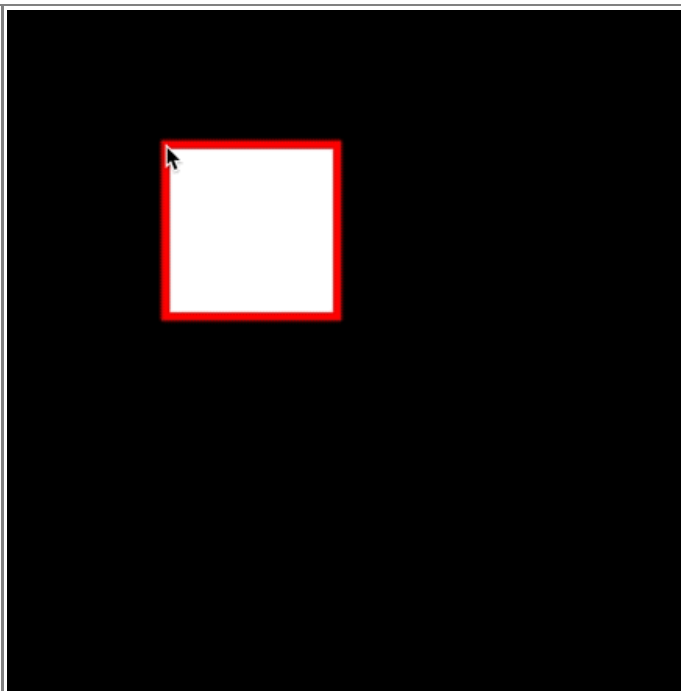
Two more useful variables available to you are **mouseX** and **mouseY**.

They contain x and y values for the mouse position while it is over the canvas and are updated every time draw is updated (60 times a second by default).

This lets us do some easy basic interaction.

The following code and comments show these in use:

```
function setup() {  
  //lets create a canvas of 400x400 pixels  
  createCanvas(400, 400);  
}  
  
function draw() {  
  //lets set the background to black  
  background(0);  
  //lets set the fill to white  
  fill(255);  
  //lets set the stroke to red  
  stroke(255, 0, 0);  
  //lets set the stroke weight to 5  
  strokeWeight(5);  
  //lets draw a rectangle that follows the mo  
  use position with a width and height of 100  
  rect(mouseX, mouseY, 100, 100);  
}
```



Scope

In programming, **scope** determines where variables and functions can be accessed from within your code. Think of it as the "visibility" or "reach" of a variable. Scope exists in plain javascript as

well, but we have not covered it yet. Scope can be complex, for the moment we will look at some limited understanding that we will need to carry on with our work.

1. Local Scope (or Function Scope):

- Variables declared inside a function (like `setup()` or `draw()` in p5.js) are only accessible within that function.
- They "disappear" or are "forgotten" once the function finishes executing.

Example:

```
function setup() {  
  let localVariable = "I'm local to the setup function!";  
  console.log(localVariable); // This works!  
  //lets create a canvas of 400x400 pixels  
  createCanvas(400, 400);  
}  
  
function draw() {  
  console.log(localVariable); // This doesn't work, the local variable is out of scope!  
}
```

2. Global Scope:

- Variables declared outside any function are accessible throughout your entire program, including inside functions.
- In p5.js, if you declare a variable at the top of your script, it's globally scoped.

Example:

```
let globalVariable = "I'm available everywhere!";  
  
function setup() {  
  console.log(globalVariable); // This works too!  
}
```

Tips:

- Be cautious with global variables; while they're accessible everywhere, overusing them can make your code harder to debug and maintain.
- If two variables have the same name, one global and one local, the **local** scope takes precedence within its function.

Understanding scope helps in organising your code, preventing errors, and ensuring variables don't unexpectedly overwrite each other.

Event functions

Similar to the event listeners we used in plain javascript, p5.js also uses events to capture user interaction. Note that unlike the interaction we have seen before, these callback functions are not bound to a specific object. There are some built-in event functions, we will look at a few:

- **mouseMoved()**

- Fired when the mouse is moved on the canvas, but no buttons are pressed.
- You can use the built-in variables *mouseX* and *mouseY* to get the location of the mouse when the event was triggered.
- <https://p5js.org/reference/p5/mouseMoved/> 
(<https://p5js.org/reference/p5/mouseMoved/>)

- **mouseDragged()**

- Fired when the mouse is moved on the canvas and a button is held down for the move.
- You can use the built-in variables *mouseX* and *mouseY* to get the mouse location when the event is triggered.
- You can use the built-in variable *mouseButton* to access the ID of the button pressed.
- <https://p5js.org/reference/p5/mouseDragged/> 
(<https://p5js.org/reference/p5/mouseDragged/>)

- **mousePressed()**

- Fired when a mouse button is pressed on the canvas.
- You can use the built-in variables *mouseX* and *mouseY* to get the mouse location when the event is triggered.
- You can use the built-in variable *mouseButton* to access the ID of the button pressed.
- <https://p5js.org/reference/p5/mousePressed/> 
(<https://p5js.org/reference/p5/mousePressed/>)

- **mouseReleased()**

- Description: Fired when a mouse button is released on the canvas.
- You can use the built-in variables *mouseX* and *mouseY* to get the mouse location when the event is triggered.
- <https://p5js.org/reference/p5/mouseReleased/> 
(<https://p5js.org/reference/p5/mouseReleased/>)

- **mouseClicked()**

- Fired when the mouse is clicked on the canvas (pressed and released).
- You can use the built-in variables *mouseX* and *mouseY* to get the mouse location when the event is triggered.
- You can use the built-in variable *mouseButton* to access the ID of the button pressed.
- <https://p5js.org/reference/p5/mouseClicked/> 
(<https://p5js.org/reference/p5/mouseClicked/>)

- **doubleClicked()**

- Fired when the mouse is double-clicked on the canvas.
- You can use the built-in variables *mouseX* and *mouseY* to get the mouse location when the event is triggered.
- You can use the built-in variable *mouseButton* to access the ID of the button pressed.
- <https://p5js.org/reference/p5/doubleClicked/> 
(<https://p5js.org/reference/p5/doubleClicked/>)

- **keyPressed()**

- Fired when any key is pressed.
- You can use the built-in variable *key* to access the key that triggers the event.
- <https://p5js.org/reference/p5/keyPressed/>  (<https://p5js.org/reference/p5/keyPressed/>)

- **keyReleased()**

- Fired when any key is released.
- You can use the built-in variable *key* to access the key that triggers the event.
- <https://p5js.org/reference/p5/keyReleased/>  (<https://p5js.org/reference/p5/keyReleased/>)

- Mobile-friendly versions of these functions include *touchStarted()* and *touchMoved()*, etc.

An example of using an event function would look like:

```
//This is a global variable, it can be accessed anywhere in the program
let globalColourValue = "red";

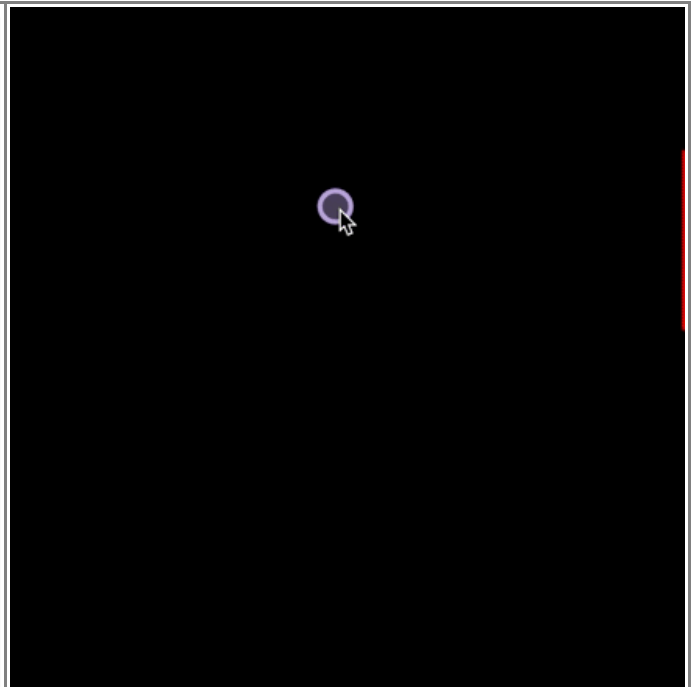
function setup() {
  //lets create a canvas of 400x400 pixels
  createCanvas(400, 400);
}

function draw() {
  //lets set the background to black
  background(0);
  //this is a local variable, it can only be accessed within the draw function
  let localColourValue = "blue";
  //lets set the fill to white
  fill(globalColourValue);
  //lets set the stroke to red
  stroke(255, 0, 0);
  //lets set the stroke weight to 5
  strokeWeight(5);
  //lets draw a rectangle that follows the mouse position with a width and height of 100
  rect(mouseX, mouseY, 100, 100);
}

function mousePressed() {
  //lets change the global variable to green when the mouse is pressed
  globalColourValue = "green";

  //if we try to access the local variable here, it will not work because it is out of scope
  //console.log(localColourValue);
}

function mouseReleased() {
  //lets change the global variable to red when the mouse is released
  globalColourValue = "red";
}
```



Other capabilities of P5.js

P5.js is a powerful tool that abstracts away much of the complexity of web programming, making it accessible and fun, especially for artists, designers, and beginners. Its capabilities extend far beyond simple drawings, offering a rich set of features for sound, video, interactivity, and more.

- **Graphics and Visualisation**

- **3D Rendering:** With the P5.js WebGL renderer, you can create 3D graphics.

- **Sound**

- **P5.sound library:** Extends P5.js to work with sound.
 - Play, analyse, and synthesise sound.

- Apply effects and filters.
- **Video & Camera**
 - **Capture Video:** Access the webcam to capture live video feeds.
 - **Play and Manipulate Videos:** Play video files and apply graphical effects or filters.
 - **Pixel Manipulation:** Access and alter individual pixels in images or video frames.
- **Other Libraries and Extensions**
 - P5.js has a growing ecosystem. There are libraries for tasks like physics simulations, typography, and even virtual reality.

4. Introducing the major Assignment

The major assignment page will be published today. Here are a few things you should know.

- You will need to work in groups, with half of the marks coming from your group work and half from your own contribution. You will need to finalise your groups by the mid semester break.
- Think of the assignment like putting together everything you have learnt in the lectures, tutorials, and the quizzes.
- There are several things that you will need to know for the assignment that have not been covered yet - but we will go through these week by week as you're developing the assignment.
- The assignment asks for a mix of creative vision and the application of the techniques we have covered in the course, both are important. Great coding or great design alone will not guarantee great marks.

That's all for today! Questions?

If you have any questions from the Lecture or Tutorial today - please head to the Ed discussions page and post your question there. When any member of the Teaching Team answers the question - this will be a resource for the whole class to learn from.

Copyright © The University of Sydney. Unless otherwise indicated, 3rd party material has been reproduced and communicated to you by or on behalf of the University of Sydney in accordance with section 113P of the Copyright Act 1968 (Act). The material in this communication may be subject to copyright under the Act. Any further reproduction or communication of this material by you may be the subject of copyright protection under the Act. Do not remove this notice.

Live streamed classes in this unit may be recorded to enable students to review the content. If you have concerns about this, please visit our [student guide \(https://canvas.sydney.edu.au/courses/4901/pages/zoom\)](https://canvas.sydney.edu.au/courses/4901/pages/zoom) and contact the unit coordinator.

[Privacy Statement \(https://www.sydney.edu.au/privacy-statement.html\)](https://www.sydney.edu.au/privacy-statement.html)